

AgenticPlace × mindX × BANKON/DAIO — Integration & Reference Guide

With x402/Algorand payments, chain mapping, Foundry→mainnet conventions, and a GNU/Tomb operational-security appendix

TL;DR

- **The stack is real and mostly open-source under one roof:** AgenticPlace (the ERC-8004 agent registry front end at `agenticplace.pythai.net`), mindX (the augmentic-intelligence backend at `github.com/abaracadabra/mindX`, mirrored `github.com/AgenticPlace/mindX`, live at `mindx.pythai.net` with a 206+ endpoint FastAPI/Swagger API), and the DAIO governance suite (`daio/` in the mindX repo, Solidity + Foundry) form a coherent pipeline. mindX writes memories to `/data`, RAGE (`rage.pythai.net`) indexes/reads them, and an AuthorAgent publishes to the RAGE page (the "Book of mindX").
- **Payments run on the x402 pattern:** "Parsec/parsec-wallet" is the user's own branding over GoPlausible's `x402-avm` Algorand implementation (the merged reference for Coinbase's x402 on the Algorand Virtual Machine); the AgenticPlace-org `pay2play-*` repos (`pay2play-algo`, `pay2play-arc`, `pay2play-glmr`) are the concrete per-request metering gateways.
- **Conventions and security:** the ecosystem develops/tests Solidity in Foundry and deploys to mainnet; the DAIO Solidity contracts live at `daio/contracts/`. For operational security of keys/secrets, **Tomb — the Crypto Undertaker** (`github.com/dyne/tomb`, `dyne.org/software/tomb/`), a LUKS/GnuPG-based GNU/Linux encrypted-volume tool, is the recommended way to protect Foundry deploy keys, `.env` secrets, DAIO governance keys, and parsec-wallet material.

Key Findings

1. **AgenticPlace is a live ERC-8004 agent registry.** `agenticplace.pythai.net` self-describes as "The living ERC-8004 agent registry. Index, mint, verify, and explore the autonomous agent economy across every chain," advertising "48 Chains," Algorand NFT minting, and "agenticORacle" (aORC) verification. Its meta keywords explicitly list "PYTHAI, mindX, Parsec Wallet, x402, OASF." `pythai` `pythai`
2. **mindX is the backend brain.** The mindX repo README describes it as "An autonomous multi-agent orchestration system implementing BDI cognitive architecture. A Godel machine ... 20 sovereign agents with cryptographic wallets, RAGE semantic search (not

RAG), DAIO governance, and dual-pillar inference (CPU + Cloud)." Live API endpoints are documented at mindx.pythai.net. [github](#)

3. **The DAIO suite is Solidity + Foundry.** mindX's third pillar is "DAIO Governance — Decentralized Autonomous Intelligence Organization. On-chain governance (Solidity + Foundry)." Solidity contracts live under [daio/contracts/](#), with README-confirmed subdirectories [inft](#), [ens](#), [THOT](#), [agentregistry](#). [github](#)
 4. **x402/Algorand payments are implemented via GoPlausible's [x402-avm](#) and the [pay2play-*](#) gateways.** Per GoPlausible's documentation README: "GoPlausible has built the reference implementation, packages, documentation and example codes for Algorand (AVM) X402 integration available as FOSS and also contributed to the Algorand Foundation PR to Coinbase's x402 protocol repo: Coinbase x402 PR #361 which has been merged." Per Genfinity (Mar 3, 2026): "On February 23, 2026, the Algorand Foundation announced that x402 protocol support reached full operational status... GoPlausible now serves as the network's designated facilitator."
 5. **Chain mapping is live and queryable.** The live agenticplace.pythai.net/api/stats endpoint returns the actual indexed-chain list keyed by EVM integer chainId.
 6. **Tomb is a real, mature GNU/Linux crypto tool.** Per the [dyne/tomb](#) GitHub repo: "Tomb is Copyright (C) 2007-2025 by the Dyne.org Foundation and maintained by Jaromil." It is suitable for protecting the ecosystem's key material.
-

Details

1. Sites, endpoints, and complete URLs

Live web properties (PYTHAI ecosystem):

- PYTHAI hub — <https://pythai.net>
- AgenticPlace — <https://agenticplace.pythai.net>
 - Browse registry — <https://agenticplace.pythai.net/browse>
 - Register/mint agent NFT — <https://agenticplace.pythai.net/register>
 - agenticORacle minter — <https://agenticplace.pythai.net/minter>
 - Algorand agents — <https://agenticplace.pythai.net/agents>
 - ChainMarketCap — <https://agenticplace.pythai.net/chainmarketcap>
 - Docs — <https://agenticplace.pythai.net/docs>
 - Stats API (JSON) — <https://agenticplace.pythai.net/api/stats>
 - Export API — <https://agenticplace.pythai.net/api/export?table=agents&format=json>

- Chain mapping page — <https://agenticplace.pythai.net/allchain.html> (referenced by the user; the live chain data is served by </api/stats> and the documented </api/allchainft> — "Blockchain connection registry metadata")
- mindX — <https://mindx.pythai.net>
 - Swagger/OpenAPI — <https://mindx.pythai.net/docs>
 - ReDoc (206+ endpoints) — <https://mindx.pythai.net/redoc>
 - Docs UI — <https://mindx.pythai.net/docs.html>
 - Book of mindX (AuthorAgent output) — <https://mindx.pythai.net/book>
 - Improvement journal — <https://mindx.pythai.net/journal>
 - Thesis evidence (JSON) — <https://mindx.pythai.net/thesis/evidence>
 - Dojo standings — <https://mindx.pythai.net/dojo/standings>
 - Inference status — <https://mindx.pythai.net/inference/status>
 - SSE activity stream — <https://mindx.pythai.net/activity/stream>
 - OpenAgents demo — <https://mindx.pythai.net/openagents>
- RAGE retrieval engine — <https://rage.pythai.net>
- BANKON — <https://bankon.pythai.net> (the BANKON vault/identity layer; "BANKON Vault — AES-256-GCM + HKDF-SHA512 encrypted credential storage" per the mindX README) ([github](#))
- Minter/permanence — minter.pythai.net
- GPT agents — <https://gpt.pythai.net>

GitHub source code (complete list):

- mindX (canonical) — <https://github.com/abaracadabra/mindX>
- mindX (org mirror, clone target) — <https://github.com/AgenticPlace/mindX>
- AgenticPlace org — <https://github.com/agenticplace>
 - mindXalpha — <https://github.com/AgenticPlace/mindXalpha>
 - mindXbeta — <https://github.com/AgenticPlace/mindXbeta>
 - mindXgamma — <https://github.com/AgenticPlace/mindXgamma>
 - mindx-self-evolution — <https://github.com/AgenticPlace/mindx-self-evolution>
 - SimpleCoder — <https://github.com/AgenticPlace/SimpleCoder>
 - openagents — <https://github.com/AgenticPlace/openagents>
 - erc-8004-contracts (fork) — <https://github.com/AgenticPlace/erc-8004-contracts>
 - pay2play — <https://github.com/AgenticPlace/pay2play>
 - pay2play-algo — <https://github.com/AgenticPlace/pay2play-algo>

- pay2play-arc — <https://github.com/AgenticPlace/pay2play-arc>
- pay2play-glmr — <https://github.com/AgenticPlace/pay2play-glmr>
- Professor Codephreak — <https://github.com/Professor-Codephreak>
- PYTHAI org — <https://github.com/pythaiml>
- RAGE/GATE org — <https://github.com/GATERAGE> (RAGE core: <https://github.com/GATERAGE/RAGE>); paper: <https://github.com/GATERAGE/RAGE/blob/main/ragepaper.md>)
- MASTERMIND — <https://github.com/mastermindML>
- automind (historical) — <https://github.com/Professor-Codephreak/automind>

DAIO contract source paths (confirmed subdirectories under [daio/contracts/](#)):

- <https://github.com/abaracadabra/mindX/tree/main/daio>
- <https://github.com/abaracadabra/mindX/tree/main/daio/contracts/inft>
- <https://github.com/abaracadabra/mindX/tree/main/daio/contracts/ens>
- <https://github.com/abaracadabra/mindX/tree/main/daio/contracts/THOT>
- <https://github.com/abaracadabra/mindX/tree/main/daio/contracts/agentregistry>
- DAIO doc — <https://github.com/abaracadabra/mindX/blob/main/docs/DAIO.md>
- Python governance — <https://github.com/abaracadabra/mindX/blob/main/daio/governance/boardroom.py> and [.../dojo.py](#)

Verification note / honesty flag: The specific contract names in the user's brief — [WarCouncil](#), [Boardroom](#) (exists as Python [boardroom.py](#), not confirmed as [.sol](#)), [DeadmansSwitch](#), [BankonIdentityRegistry](#), [X402AccessGate](#), [ChainRegistry](#), and a [Daio](#) core contract — could **not** be independently confirmed as [.sol](#) files from publicly fetchable pages; only [inft](#), [ens](#), [THOT](#), and [agentregistry](#) are named in the README under [daio/contracts/](#). These contracts may exist in the unfetched tree, but readers should verify directly in [daio/contracts/](#). Also note a license discrepancy: the mindX repo README states **MIT License**, not Apache-2.0; the "cypherpunk2048/Apache-2.0" convention should be verified per-file via SPDX headers. [github](#)

2. The mindX → RAGE publishing flow

Per the RAGE architecture ([github.com/GATERAGE](#)), memory is treated as an append-only event stream ("Logs are memory"). The integration works as follows:

1. **mindX writes memories to [/data](#)** — mindX runs 5-minute improvement cycles; agent outputs, decisions, and dialogues are persisted (159,000+ memories, 132,000+ embeddings)

in pgvector per the README). ([github](#))

2. **RAGE indexes and reads them back** — RAGE ingests via ([POST /ingest](#)), producing capsules (chunks) indexed with a hybrid vector (([pgvector scale](#))/DiskANN) + lexical (BM25) index; retrieval is via ([POST /query](#)). ([GitHub](#))
3. **An AuthorAgent publishes from memory to the RAGE page** — the "Book of mindX" at ([mindx.pythai.net/book](#)) is "written by AuthorAgent," and content promotes through integrity tiers (L0 Draft → L1 Hashed → L2 CID → L3 Signed → L4 Validated → L5 Promoted) with wallet-signed receipts and optional IPFS/CID permanence anchored via ([minter.pythai.net](#)). ([github](#))

RAGE's MVP API surface: ([POST /ingest](#)), ([POST /query](#)), ([POST /attest](#)), ([POST /validate](#)), ([POST /promote/prepare](#)), ([POST /promote/finalize](#)). Receipts follow a chain-agnostic canonical JSON payload (domain/action/subject/content_hash/namespace/nonce/ts/exp), canonicalized (sorted keys, UTF-8), hashed, and signed. ([GitHub](#))

3. DAIO deployment (current, mainnet)

The DAIO ("Decentralized Autonomous Investment/Intelligence Organization") governance suite is Solidity-on-Foundry. The mindX CEO Agent receives "DAIO governance directives (on-chain → off-chain bridge)," which flow to the MastermindAgent → CoordinatorAgent → 20+ specialized agents. Governance consensus is handled by the Boardroom (CEO + 7 soldiers, weighted voting, dissent branches) and reputation by the Dojo (7 ranks, Novice → Sovereign). Contracts build on OpenZeppelin and are developed/tested with Foundry. ([github](#))

Chain targeting: DAIO deploys to EVM mainnets. The ERC-8004 registry contract referenced on AgenticPlace is ([0x8004a169fb4a3325136eb29fa0ceb6d2e539a432](#)) (viewable on Celo Blockscout), consistent with the canonical ERC-8004 registry address ([0x8004...9432](#)). The user's stated deployment posture is "happening now" on mainnet, aligned with the house convention of direct mainnet deployment (below). ([pythai](#))

4. x402 payment protocol + Algorand (GoPlausible x402-avm) + Parsec

The x402 standard revives HTTP 402 "Payment Required." Per Coinbase's launch page, x402 "resurrect[s] the HTTP 402 'Payment Required' status code, a dormant feature of the web designed for seamless payment requests" — the code was reserved in the 1990s but never implemented until this protocol. A resource server responds ([402](#)) with payment requirements; the client constructs a signed payment and retries; a facilitator verifies and settles on-chain. Per Eco's technical primer: "Coinbase open-sourced x402 in May 2025, then donated it to the x402 Foundation at the Linux Foundation on April 2, 2026." It originated in the EVM/Coinbase context and was extended to Algorand.

GoPlausible x402-avm (the reference under "Parsec"):

- Docs root — (<https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/README.md>)
- Python examples — ([../python/x402-avm-avm-examples-python.md](#)), ([../python/x402-avm-fastapi-examples-python.md](#)), ([../python/x402-avm-httpx-examples-python.md](#)), ([../python/x402-avm-requests-examples-python.md](#)), ([../python/x402-avm-extensions-examples-python.md](#))
- TypeScript examples — ([../typescript/x402-avm-express-examples.md](#)), ([../typescript/x402-avm-next-examples.md](#)), ([../typescript/x402-avm-hono-examples.md](#)), ([../typescript/x402-avm-avm-examples.md](#))
- Provenance: "GoPlausible has built the reference implementation, packages, documentation and example codes for Algorand (AVM) X402 integration available as FOSS and also contributed to the Algorand Foundation PR to Coinbase's x402 protocol repo: Coinbase x402 PR #361 which has been merged." Since Feb 23, 2026, GoPlausible is the Algorand network's designated x402 facilitator. ([GitHub](#))
- Install: (`pip install "x402-avm[avm]"`) (Python; imports as (`from x402...`)); npm packages (`@x402-avm/avm`), (`@x402-avm/express`), (`@x402-avm/next`), (`@x402-avm/hono`), (`@x402-avm/core`). (Note: as of v2.6+, (`algosdk`) is no longer a direct dependency — packages use (`@algorandfoundation/algokit-utils`).)
- Key constants: (`ALGORAND_MAINNET_CAIP2 = "algorand:wGHE2Pwvdv7S12BL5Fa0P20EGYesN73ktiC1qzkkit8="`), (`ALGORAND_TESTNET_CAIP2 = "algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI="`). Default facilitator: (<https://x402.org/facilitator>). ([GitHub](#)) ([GitHub](#))
- Mechanism: Algorand "exact" scheme uses atomic transaction groups (ASA/ALGO transfer) with pooled fees for fee abstraction; USDC ASA IDs are exposed as constants (`(USDC_MAINNET_ASA_ID, USDC_TESTNET_ASA_ID)`).

Clarification (honesty flag): "Parsec" / "parsec-wallet" is the user's own branded abstraction; there is no separately-published public (`parsec-wallet`) library found (unrelated GitHub hits like (`artplant/parsec-contracts`) and (`parsecular/parsec-mcp`) are different projects). The AgenticPlace (`register`) page lists "Pera / Parsec" as wallet options, confirming Parsec is a wallet integration in the user's stack. The underlying reference implementation to cite is GoPlausible (`x402-avm`). Note also that the AgenticPlace-org (`pay2play-algo`) repo is a **standalone** gateway that does NOT depend on GoPlausible packages — it vendors an agnostic core from (`pay2play-arc`) and implements Algorand atomic-group settlement with an (`X-Algo-Payment`) header on an "x402-shaped" HTTP surface.

The pay2play family (per-request metering gateways):

Repo	Chain	Asset	Settlement	Header
<code>pay2play</code>	Circle Arc	USDC	Circle Gateway batched (gasless)	—
<code>pay2play-arc</code>	Arc Testnet (EVM chainId 5042002)	USDC (6-dec ERC-20)	EIP-3009 + Circle Gateway	<code>payment-signature</code>
<code>pay2play-algo</code>	Algorand testnet	ALGO (6-dec native)	Atomic group + ApplicationCall (<code>PaymentMeter.algo.ts</code> , PuyaTS)	<code>X-Algo-Payment</code>
<code>pay2play-glmr</code>	Moonbeam (GLMR)	USDC.wh	EVM, per-request	—

5. Chain mapping (live data from `/api/stats`)

The AgenticPlace homepage advertises "48 Chains"; the live `agenticplace.pythai.net/api/stats` JSON returns the actual indexed-chain set (**43 chains that currently hold indexed agents**) keyed by **integer EVM chainId** with schema `{chain_id, cnt, name, short_name}`. Highest-volume entries (chainId — name — agent count):

- 56 — BNB Smart Chain — 58,372
- 8453 — Base — 25,767
- 45056 — Billions — 15,706
- 1 — Ethereum Mainnet — 14,764
- 143 — Monad — 8,344
- 4326 — MegaETH Mainnet — 8,246
- 42220 — Celo Mainnet — 6,796
- 84532 — Base Sepolia — 4,861
- 100 — Gnosis — 3,389
- 11155111 — Ethereum Sepolia — 3,149
- 5042002 — Arc Network Testnet — 2,041
- 42161 — Arbitrum One — 774
- 10 — OP Mainnet — 484
- 137 — Polygon — 329
- 43114 — Avalanche C-Chain — 167
- 1776 — Injective — 165
- 1088 — Metis Andromeda — 138

- 196 — X Layer — 128
- 2741 — Abstract — 118
- 59144 — Linea — 109
- 534352 — Scroll — 107
- 5000 — Mantle — 92
- 167000 — Taiko Alethia — 90

(Plus testnets/smaller chains: Shape 360, SKALE Base 1187947933, Hedera Testnet 296, GOAT 2345, Amoy 80002, and others.) The separate `/chainmarketcap` page ("123 ALLCHAIN from A-Z") is a broader "2500+ EVM networks" directory. Aggregate stats at capture: `total_local` 160,668 agents, `total_remote` 223,940, `daily_new` 3,623, `x402`-enabled 12,435, MCP agents 8,539, A2A agents 7,270.

CAIP-2 note: the `/api/stats` payload uses raw integer chainIds, not CAIP-2 strings. For cross-chain identity map to CAIP-2 as needed: EVM = `eip155:<chainId>` (e.g. `eip155:1`, `eip155:8453`, `eip155:5042002`); Algorand mainnet = `algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYEsN73k` (CAIP-2 truncated genesis) or the x402-avm full form `algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYEsN73ktiC1qzkkIt8=`. [GitHub](#) [GitHub](#)

6. Foundry-for-testing, mainnet-for-deployment convention

The mindX stack confirms "On-chain governance (Solidity + Foundry)" and lists Foundry (github.com/foundry-rs/foundry) and OpenZeppelin among dependencies. The "cypherpunk2048" house standard as described by the user: Apache-2.0 license, no admin keys post-deploy, no upgradeable proxies, mainnet-only (no testnets), flat snake_case naming, Solidity 0.8.26, Python ≥3.12, Podman over Docker. Foundry toolchain: `forge` (build/test), `cast` (chain interaction), `anvil` (local node), `chisel` (REPL). Typical deploy: `forge script script/Deploy.s.sol --rpc-url $MAINNET_RPC --private-key $PK --broadcast --verify`. (Solidity 0.8.26, released May 21, 2024, notably adds custom errors in `require` and improved Yul optimizer sequencing.) [github](#) [CryptoSkills](#)

Example Integration (protocol-agnostic guide)

A generalized pattern that applies beyond this one stack — an agentic marketplace + memory + governance + micropayment loop:



Step-by-step (agnostic):

1. **Identity:** An agent registers in the registry (ERC-8004 `IdentityRegistry`) and/or mints an identity NFT (on AgenticPlace, an Algorand ARC-69 aNFT/dNFT/iNFT/THOT via Pera or Parsec wallet). Portable identifier keyed by chain (CAIP-2 or integer chainId).
2. **Memory write:** The agent persists reasoning/outputs to a local durable store (`/data`), append-only ("logs are memory").
3. **Retrieval/index:** A retrieval engine (RAGE) ingests and builds a hybrid semantic+lexical index; queries return capsules with provenance metadata (CID + signature).
4. **Publish:** An AuthorAgent synthesizes memory into a public artifact and promotes it through integrity tiers, optionally anchoring to IPFS with wallet-signed receipts.
5. **Monetize (x402):** A consumer hits a paid endpoint → server returns `402` with payment requirements (scheme `exact`, network, `payTo`, price) → client signs an Algorand atomic group (or EVM EIP-3009) → facilitator verifies/settles → resource returned. Use `@x402-avm/*` (TS) or `x402-avm[avm]` (Python), or the standalone `pay2play-*` gateways.
6. **Govern:** Treasury/parameter decisions route through DAIO Solidity contracts (Boardroom consensus), developed and tested in Foundry, deployed to mainnet.

Minimal x402-avm server (Python/FastAPI, protocol-agnostic):

```
python

# pip install "x402-avm[fastapi,avm]"
from x402.http import PaymentOption
from x402.http.types import RouteConfig
AVM = "algorand:SG01GKSzyE7IEPItTxCByw9x8FmnrCDexi9/c0UJ0iI=" # testnet CAIP-2
routes = {
    "GET /premium": RouteConfig(
        accepts=[PaymentOption(scheme="exact", pay_to="YOUR_ALGO_ADDR",
                                price="$0.01", network=AVM)],
        description="Premium data")}
```

Minimal x402-avm client (auto-pays 402s):

```
python

from x402 import x402Client
from x402.mechanisms.avm.exact import register_exact_avm_client
client = x402Client()
register_exact_avm_client(client, signer) # signer holds the ALGO key
resp = await client.fetch("https://api.example.com/paid-resource")
```

GNU Tools Operational-Security Appendix — Tomb, the Crypto Undertaker

This section is a self-contained "agnostic guide to GNU tools" for securing key material in a decentralized-infrastructure context, focused on **Tomb**.

What Tomb is

Tomb ("the Crypto Undertaker") is a free/open-source command-line tool for encrypting and hiding files on GNU/Linux, maintained by the **Dyne.org Foundation** and lead developer **Denis "Jaromil" Roio**. Per the [dyne/tomb](https://github.com/dyne/tomb) GitHub repo: "Tomb is Copyright (C) 2007-2025 by the Dyne.org Foundation and maintained by Jaromil"; the AUTHORS file notes "Tomb is designed and written by Denis Roio jaromil@dyne.org." It derives from scripts used in the dyne:bolic 100% Free GNU/Linux distribution. Per the project: "Tomb aims to be a free and open source system for easy encryption and backup of personal files, written in code that is easy to review and links well reliable GNU/Linux components." It is "a simple shell script (Zsh) using standard filesystem tools (GNU) and the cryptographic API of the Linux kernel (cryptsetup and LUKS)." A "tomb" is an encrypted LUKS volume (a `.tomb` file) unlocked by a separate key file (`.tomb.key`), GnuPG-encrypted with a password), enabling key/storage physical separation. [GitHub](https://github.com/dyne/tomb) [GitHub](https://github.com/dyne/tomb)

Official URLs

- Homepage — `(https://www.dyne.org/software/tomb/)` (also `(https://dyne.org/tomb/)`)
- GitHub — `(https://github.com/dyne/tomb)`
- Signed stable releases (use in production) — `(https://files.dyne.org/tomb)`
- Manual/docs — `(man tomb)` and `(https://dyne.org/docs/tomb)`

Dependencies

- Core: **zsh** (Tomb is a Zsh script), **cryptsetup** (LUKS), Linux kernel **dm-crypt**, **GnuPG** (key encryption), **pinentry** (password prompts), plus standard GNU tools (`(sudo)`, `(mkfs)`, `(losetup)`).
- Optional: **steghide** (hide a key inside a JPEG/image via steganography), **argon2** (KDF), FIDO2 hardware key (via `(hmac-secret)`).
- Companion tools: `(pass-tomb)` (wraps the `(pass)` password manager inside a tomb, written in Bash), zuluCrypt (C++ GUI), Mausoleum (Python GUI), Secrets (splits a Tomb key into shares distributed to peers).

Command reference

- `(tomb dig -s <MB> secret.tomb)` — create (`dig`) an empty tomb volume of a given size.
- `(tomb forge secret.tomb.key)` — forge (generate) a new key, GnuPG-password-protected.
- `(tomb lock secret.tomb -k secret.tomb.key)` — format/lock the tomb with the key (LUKS).
- `(tomb open secret.tomb -k secret.tomb.key)` — open (mount) the tomb as a normal folder.
- `(tomb close)` — unmount a tomb; `(tomb slam)` — force-close immediately even if files are busy.
- Extras: `(tomb bury)`/`(tomb exhume)` (steganographically hide/recover a key inside an image), `(tomb passwd)` (change key password), `(tomb list)`/`(tomb index)`/`(tomb search)` (manage/search open tombs).

Canonical quick-start sequence (from the project README):

```
$ tomb dig -s 100 secret.tomb
$ tomb forge secret.tomb.key
$ tomb lock secret.tomb -k secret.tomb.key
$ tomb open secret.tomb -k secret.tomb.key
# ... use the mounted folder ...
$ tomb close          # or: tomb slam
```

Security guidance from the project: "do not use the latest Git version in production environments, but use a stable release versioned and packed as tarball on files.dyne.org." Keys and storage should be kept on separate media (e.g. key on a USB stick), so one always needs both the tomb and the key plus its password.

How Tomb fits the DAIO/BANKON ecosystem (operational security)

Because the ecosystem's threat surface is private keys and secrets — Foundry deploy keys, DAIO governance signer keys, agent wallet keys, and x402/parsec-wallet mnemonics — Tomb keeps them encrypted-at-rest and only mounted during the exact moment of use:

- **Foundry private keys:** keep `(PRIVATE_KEY)` / keystore JSON inside a tomb; `(tomb open)` immediately before `(forge script ... --broadcast)`, then `(tomb close)`. Combine with Foundry's own `(cast wallet import)`/keystore for a second layer.
- **`(.env)` secrets:** store project `(.env)` (RPC URLs, API keys such as `(GEMINI_API_KEY)`, `(GROQ_API_KEY)`, `(OPENAI_API_KEY)`) inside a tomb; bind-mount into the working directory only while running.
- **DAIO governance keys:** the multi-signer keys for Boardroom/treasury actions live in a tomb whose key is itself split (via Secrets) among signers, aligning with multi-party governance.
- **parsec-wallet / Algorand mnemonics:** the `(ALGO_MNEMONIC)` used by `(pay2play-algo)` deploy scripts and Parsec wallet seed material are ideal tomb contents; never leave them in plaintext `(.env.example)` overrides.
- **Deployment artifacts:** broadcast logs and `(deployments/*.json)` with sensitive addresses can be archived inside a tomb for tamper-evident cold storage.

Complementary GNU/hardening tools worth pairing with Tomb: **GnuPG** (sign releases/commits, encrypt to recipients), **pass + pass-tomb** (structured secret tree inside a tomb), **age/sops** (modern file/secret encryption for CI), **ufw** (firewalling inference/API nodes; mindX ships a "doubletap" ufw script), and **Podman** (rootless containers, the ecosystem's Docker replacement) to sandbox agent execution.

Recommendations

1. **Start from the canonical source of truth.** Clone `github.com/AgenticPlace/mindX` (mirror of `abaracadabra/mindX`) and read `docs/NAV.md`, `docs/DAIO.md`, `docs/TECHNICAL.md`, and `openagents/docs/JUDGE_TOUR.md`. Verify the exact `daio/contracts/` tree yourself — do not assume `WarCouncil`/`X402AccessGate`/`ChainRegistry` exist as `.sol` until you see them; only `inft`, `ens`, `TH0T`, `agentregistry` are publicly confirmed.
2. **Wire payments through GoPlausible x402-avm.** Use `@x402-avm/*` or `x402-avm[avm]` with the CAIP-2 network constants above; treat "Parsec/parsec-wallet" as your local wallet-integration name over that reference. For non-Circle Algorand metering, the standalone `pay2play-algo` gateway is the closest matching pattern. Since GoPlausible is now the Algorand designated facilitator, you can point at the public facilitator or run your own.
3. **Reconcile the license/convention before mainnet.** The repo README says MIT while the house standard is stated as Apache-2.0 — pick one and make SPDX headers consistent. Confirm the Solidity 0.8.26 pragma and no-proxy/no-admin-key posture per contract; the "no admin keys post-deploy" claim is the single highest-value thing to verify since it is irreversible.
4. **Adopt Tomb immediately for key hygiene.** Install the signed release from `files.dyne.org/tomb`; move all deploy keys, `.env`, and mnemonics into tombs with keys on separate USB media; script `tomb open`/`tomb close` around `forge` broadcasts.
5. **Staged thresholds that change the plan:**
 - *If a formal audit is planned:* freeze Foundry test coverage $\geq 90\%$ on `daio/contracts/` before any mainnet broadcast.
 - *If governance keys are shared among >2 signers:* move from a single tomb to Secrets-split key shares (and/or a Safe multisig, given no upgradeable proxies).
 - *If x402-enabled agents (currently 12,435) come to dominate traffic:* deploy a dedicated facilitator rather than relying on the public `x402.org/facilitator`.
 - *If deploying immutably (no admin keys):* run a full mainnet-fork dry run in `anvil` first, since there is no post-deploy fix.

Caveats

- Several PYTHAI properties are personal/niche; some GitHub subtrees (`daio/contracts/*`, `docs/DAIO.md`) and `allchain.html` were not directly fetchable, so a few specifics (exact `.sol` filenames, per-file SPDX/pragma, `foundry.toml` presence) are inferred from the README and the live API and must be verified in-repo.

- The user-supplied contract names (`WarCouncil`, `DeadmansSwitch`, `BankonIdentityRegistry`, `X402AccessGate`, `ChainRegistry`, `Daio` core) are not publicly confirmed as `.sol` files; treat them as the intended/authoritative design per the user, and cross-check against the live tree. `Boardroom` and `Dojo` exist as Python governance modules under `daio/governance/`.
- "48 chains" (marketing headline) vs 43 chains with live indexed agents (from `/api/stats`) vs "2500+" (chainmarketcap directory) are three different datasets — cite the one that matches your use. These counts are live and will drift over time.
- "DAIO" is overloaded: here it is PYTHAI's governance suite (the README expands it as "Decentralized Autonomous Intelligence Organization"; the user's brief expands it as "Decentralized Autonomous Investment Organization"), not the unrelated NASDAQ ticker DAIO (Data I/O Corp) or `tonyin/the-daio`.
- Tomb requires root/sudo and is Linux-only; it is not a substitute for hardware wallets for high-value signing keys — use it alongside, not instead of, HSM/hardware custody.
- x402 governance moved to the Linux Foundation's x402 Foundation (Apr 2, 2026); the protocol surface may continue to evolve, so pin package versions (`@x402-avm` v2.6+ dropped the `algosdk` direct dependency, a breaking change).